BUILD THIS APP: SUNO PROMPT WORKSHOP

You are building a complete, functional frontend web application.

The product is a fast, structured music-prompt building workshop for creating high-quality **Suno Style Prompts**.

This is not a generic AI prompt generator.

The user should be able to visually assemble musical concepts, understand unfamiliar terminology through Traditional Chinese explanations, and immediately receive a well-structured English prompt suitable for Suno.

---

1. SOURCE MATERIALS

Two sources are provided to you:

Source 1 — This Build Prompt

This is the **authoritative specification for the product**.

It defines:

- Product behavior
- UX
- UI
- Architecture
- MVP scope
- Prompt-generation logic
- Data structure
- Feature priorities

Follow this specification when making product decisions.

Source 2 — Attached PDF:

`AI Music prompt generator 概念書.pdf`

This is the **reference knowledge base for the music content**.

Use it for:

- Genre taxonomy
- Music terminology
- Genre characteristics
- Instrument vocabulary
- Texture vocabulary
- Mood vocabulary
- Era concepts
- Rhythm / groove terminology
- Prompt vocabulary
- Suno-oriented examples
- Natural-language explanations
- Genre-specific seed data

The PDF is the source of truth for the underlying music knowledge.

Do not invent an unrelated music taxonomy when the PDF already provides the relevant structure.

If the Build Prompt and PDF differ on **product behavior or UX**, follow this Build Prompt.

If you need music terminology or genre content, use the PDF as the primary reference.

---

2. PRODUCT GOAL

Create a web app where a user can:

1. Browse music concepts.
2. Understand what those concepts mean.
3. Click concepts to add them to a Prompt Builder.
4. Combine genres, moods, textures, eras, instruments, rhythms, techniques, etc.
5. Let the system determine an intelligent prompt order.
6. Receive a clean English Suno Style Prompt.
7. See a structured explanation of how the prompt was assembled.
8. Copy the prompt.
9. Save prompts to History.
10. Save reusable configurations as Favorites / Templates.
11. Receive soft warnings when selected concepts may conflict.

The experience must be fast.

Do NOT use a step-by-step wizard.

The user should be able to experiment freely.

---

3. CORE PRODUCT PRINCIPLE

Do not build this as:

> "A collection of music keywords."

Build it as:

> "A structured music-prompt workshop that understands relationships between musical concepts."

The system should understand concepts such as:

- Primary Genre
- Genre Influence
- Mood / Vibe
- Era
- Texture / Timbre
- Instrumentation
- Rhythm / Groove
- Technique
- BPM / Dynamics

The generated prompt should preserve hierarchy rather than simply concatenating every selected word.

---

4. MVP SCOPE

INCLUDE

- Suno Style Prompt generation
- 30 major genre categories
- Hierarchical music database
- Level 2 default browsing
- Optional Level 3 expansion
- Primary Genre
- Genre Influences
- Mood / Vibe
- Era
- Texture / Timbre
- Instruments
- Rhythm / Groove
- Technique
- BPM / Dynamics

- Automatic prompt ordering
- Weighted prompt logic
- Soft conflict warnings
- Structured preview
- Editable final prompt
- Copy to clipboard
- History
- Favorites / Templates
- LocalStorage persistence
- Curated starter presets
- Responsive UI

DO NOT INCLUDE

- User accounts
- Authentication
- Backend
- Cloud database
- Payments
- AI API calls
- LLM-powered generation
- Natural-language-to-prompt generation
- Lyrics generation
- Lyrics Structure
- Udio support
- Other music-platform outputs
- Collaboration
- Cloud synchronization

Do not add these features "because they might be useful."

Keep the MVP focused.

---

5. UI LANGUAGE

The application UI should primarily use **Traditional Chinese**.

Professional musical terminology may remain in English.

Examples:

- Genre
- Mood
- Texture

- Timbre
- Swing
- Syncopation
- Rhodes
- Walking Bass
- BPM

Every technical concept should have a concise Traditional Chinese explanation.

Example:

Syncopation

> 刻意強調非預期的拍點，讓節奏產生跳動或錯位感。

The generated Suno Style Prompt itself must be in English.

---

6. VISUAL DIRECTION

The visual language should feel like a modern **DAW / music-production workspace**.

Think:

- Dark interface
- Creative tool
- Dense but organized
- Cards
- Tags
- Panels
- Compact controls
- Strong hierarchy
- Clear state changes

Avoid:

- Generic corporate SaaS appearance
- Excessive gradients
- Excessive glassmorphism
- Huge marketing sections
- Excessive animation
- "AI magic" visual clichés

The application should feel like a tool someone could actually use while making music.

Priority

1. Functionality
2. Speed
3. Clarity
4. Prompt quality
5. Visual polish

---

7. MAIN SCREEN

Use a desktop-first three-column workspace.

LEFT — MUSIC PALETTE

Browse available concepts.

Categories should include:

- Genre
- Mood / Vibe
- Era
- Texture / Timbre
- Instruments
- Rhythm / Groove
- Technique
- BPM / Dynamics

The user clicks an item to add it to the Builder.

Do not force sequential selection.

CENTER — PROMPT BUILDER

Show the user's selected concepts as cards / chips.

Each selection should display:

- English name
- Traditional Chinese explanation
- Category
- Remove control

The Builder should make the current musical direction immediately understandable.

RIGHT — OUTPUT

Show:

Structured Prompt

For example:

Primary Genre
Jazz

Influence
Rock

Mood
Melancholic

Era
1970s

Texture
Warm Analog Tape

Instrumentation
Rhodes, Tenor Saxophone, Walking Bass

Tempo
90 BPM

Then show:

Suno Style Prompt

Jazz with rock influence, melancholic, 1970s-inspired production, warm analog tape texture, Rhodes piano, tenor saxophone, walking bass, 90 BPM.

Provide a prominent:

Copy

button.

---

8. DATABASE ARCHITECTURE

Do NOT hard-code the database directly into individual UI components.

Create a structured and extensible music knowledge model.

A concept should support properties similar to:

```
{
  id: "warm-analog-tape",
  category: "texture",
  name: "Warm Analog Tape",
  nameZh: "溫暖類比磁帶",
  descriptionZh: "帶有類比磁帶般的溫暖、柔和、輕微飽和與顆粒感。",
  promptTerms: ["warm analog tape texture"],
  level: 2,
  parentId: null,
  children: []
}
```

The exact implementation may be improved, but the architecture must support:

- categories
- hierarchy
- parent/child relationships
- Traditional Chinese descriptions
- English prompt terms
- metadata
- future expansion

---

9. HIERARCHY

The database must support at least three levels.

LEVEL 1

Broad concept.

Example:

Jazz

LEVEL 2

Common characteristics.

Example:

Swing
Walking Bass
Improvisation
7th Chords

LEVEL 3

More specific variations.

Example:

Piano
→ Rhodes
→ Warm Electric Rhodes

The default interface should show **Level 2**.

Level 3 should be available through expansion.

Do not force users to go deeper.

The user decides how precise the prompt should become.

---

10. GENRE DATABASE

Implement all 30 major genre categories from the provided PDF:

1. Pop
2. Hip-Hop / Rap
3. R&B / Soul
4. Dance / Club
5. Electronic / EDM
6. Rock
7. Metal
8. Punk
9. Underground / Independent
10. Jazz
11. Classical
12. Country
13. Folk / Singer-Songwriter
14. Retro / Vintage
15. Latin
16. Reggae / Ska
17. African
18. Asian - Indian / South Asian

19. Middle Eastern
20. East Asian
21. Southeast Asian
22. Ambient / Atmospheric
23. Study / Focus
24. Workout / Gym
25. Gaming / Video Game
26. Soundtrack / Score
27. Religious / Gospel
28. Children's
29. Experimental / Avant-Garde
30. Viral / Internet

Use the PDF to populate their representative sonic characteristics and terminology.

Do not create extremely deep data for every genre.

Prioritize useful representative concepts.

---

11. REPRESENTATIVE DATABASE DEPTH

The MVP should provide enough seed data to make every major genre useful.

For each genre, aim to provide representative entries across relevant dimensions such as:

- Characteristics
- Mood
- Instrumentation
- Rhythm
- Texture
- Production / Era
- Technique

Not every genre needs every dimension.

Use the PDF to determine what is musically appropriate.

The architecture must allow more entries to be added later without changing the application logic.

---

12. PRIMARY GENRE LOGIC

When the user selects a Genre:

The first selected Genre becomes the default **Primary Genre**.

If the user selects additional Genres, they become **Influences**.

Example:

Selected:

> Jazz
> Rock
> Electronic

Default:

> Primary: Jazz
> Influences: Rock, Electronic

The user must be able to change which Genre is Primary.

Do not create a complicated "fusion mode" workflow.

Speed is more important.

---

13. INFLUENCE LANGUAGE

When a secondary genre is used, the generated prompt should communicate that it is an influence rather than an equal competing genre.

Example:

Primary:

> Jazz

Influence:

> Rock

Preferred output:

> Jazz with rock influence...

Avoid blindly generating:

    Jazz, Rock

because that loses hierarchy.

The exact grammatical phrasing can be adapted to make the English natural.

---

14. PROMPT ENGINE

Implement a deterministic, rule-based prompt engine.

Do not use an external AI API.

The engine should be modular and easy to modify.

Use configurable category weights.

Initial weighting:

Primary Genre 100
Genre Influence 90
Core Style 80
Mood / Vibe 70
Era 60
Texture / Timbre 55
Instrumentation 50
Rhythm / Groove 45
Technique 40
BPM / Dynamics 35

These are initial weights, not immutable rules.

The system should be designed so the weights can be changed later from one configuration source.

---

15. PROMPT ORDER

The final prompt should generally follow this hierarchy:

1. Primary Genre
2. Genre Influence

3. Core Style
4. Mood / Vibe
5. Era
6. Texture / Timbre
7. Instrumentation
8. Rhythm / Groove
9. Technique
10. BPM / Dynamics

Do not simply sort alphabetically.

Do not sort by the order the user clicked.

The purpose of the engine is to create a coherent musical instruction.

---

16. DUPLICATE HANDLING

Avoid obvious repetition.

For example:

> Warm
> Warm Analog Tape
> Warm Vintage Texture

should not blindly produce all three if they are semantically redundant.

However, do not over-engineer semantic similarity detection in the MVP.

Implement simple deterministic duplicate / redundancy rules where practical.

---

17. USER CONTROL

The system determines prompt order automatically.

The user should NOT have to manually arrange the final prompt.

However, the user can control:

- Which Genre is Primary
- Which concepts are selected
- Which Level 3 concepts are expanded / selected
- Whether to remove concepts

- Manual editing of the final generated prompt

---

18. MANUAL OUTPUT EDITING

The final Suno Style Prompt should be editable.

When the user manually edits it:

- Keep the structured Builder unchanged.
- Treat the edited prompt as a custom version.
- Do not attempt full two-way synchronization.

Provide:

Reset to Generated Prompt

to restore the engine-generated version.

---

19. CONFLICT DETECTION

Implement lightweight conflict detection.

The philosophy:

> WARN, DON'T BLOCK.

Never prevent the user from generating a prompt because of a conflict.

Never silently modify the user's selections.

---

20. CONFLICT TYPES

Implement initial rules for:

A. Semantic conflicts

Example:

> Slow Ballad + 180 BPM

B. Energy / style conflicts

Example:

> Ambient + extremely aggressive punk energy

C. Redundancy

Example:

> Warm + Warm Analog Warmth

D. Overloaded prompts

If too many competing concepts are selected, warn the user.

Example Traditional Chinese warning:

> ⚠️ 目前的 Prompt 包含較多互相競爭的方向，可能會讓生成結果變得不穩定。

The warning should be dismissible or ignorable.

Do not block generation.

---

21. OUTPUT FORMAT

The user should see two representations.

Structured representation

Shows how the prompt was assembled.

Final representation

A clean English Suno Style Prompt.

The Copy button should copy ONLY the final prompt.

Do not include labels such as:

> Genre:
> Mood:
> Texture:

inside the copied prompt.

---

22. HISTORY

Use LocalStorage.

Store the **30 most recent copied prompts**.

When the user clicks Copy:

1. Copy the final prompt.
2. Show immediate confirmation.
3. Create a History entry.

Each History entry should include:

- id
- timestamp
- generated prompt
- structured selections
- custom prompt state if applicable

Users should be able to:

- Load
- Copy
- Delete

History must survive page refresh.

---

23. FAVORITES / TEMPLATES

Users can save a Builder configuration as a Favorite / Template.

When saving, allow a custom name.

Example:

> Rainy Tokyo Jazz

A Favorite should preserve:

- selected concepts
- Primary Genre
- Influences

- generated prompt
- relevant configuration state

Users can:

- Load
- Edit
- Rename
- Delete
- Customize the resulting prompt

Favorites should use LocalStorage.

---

24. STARTER PRESETS

Provide curated starter examples on the initial screen.

Examples:

- Late Night Jazz
- Rainy Day Piano
- 80s Cinematic Synth
- Aggressive Workout Rock
- Cyberpunk Electronic
- Warm Indie Folk

Clicking one should load its configuration into the Builder.

These are starting points, not fixed prompts.

---

25. EMPTY STATE

The empty Builder should communicate:

> Start building your sound.

Do not show a giant marketing hero.

Keep the workspace immediately usable.

Starter presets should be visible without obstructing the main workflow.

---

26. COPY FEEDBACK

When copied:

Show a small immediate state change such as:

> Copied!

Do not open a modal.

---

27. CLEAR / RESET

Provide:

Clear Builder

Removes all structured selections.

Reset Generated Prompt

Restores the system-generated prompt after manual editing.

These are different operations.

---

28. RESPONSIVE DESIGN

Desktop is the primary experience.

On smaller screens, collapse the three-column workspace into a logical stacked or tabbed workflow:

1. Palette
2. Builder
3. Output

Do not force the desktop layout onto mobile.

---

29. LOCALIZATION RULE

UI text:

Traditional Chinese.

Music terminology:

English where appropriate.

Explanations:

Traditional Chinese.

Generated Suno prompt:

English.

Do not use Simplified Chinese anywhere in the UI.

---

30. TECHNICAL STRUCTURE

Keep these systems separated:

Components

UI and interaction.

Music Database

All genre and music knowledge.

Prompt Engine

Selection → weighted ordering → prompt construction.

Conflict Engine

Selection → warnings.

Persistence Layer

History and Favorites via LocalStorage.

Preset Data

Starter configurations.

Do not place all business logic inside React components.

---

31. SUGGESTED COMPONENT STRUCTURE

Use a modular structure similar to:

```
App
├── Workspace
│   ├── MusicPalette
│   │   ├── CategoryTabs
│   │   ├── Search
│   │   └── ConceptList
│   │
│   ├── PromptBuilder
│   │   ├── SelectedConcepts
│   │   ├── PrimaryGenreControl
│   │   └── ConflictWarnings
│   │
│   └── PromptOutput
│       ├── StructuredPreview
│       ├── PromptEditor
│       └── CopyButton
│
├── Presets
├── History
└── Favorites
```

This is a suggested structure, not a requirement to use these exact component names.

---

32. SEARCH / DISCOVERY

Because the database may become large, include a lightweight search field in the Music Palette.

Search should search:

- English name
- Traditional Chinese name

- Description
- Relevant prompt terms

Example:

Searching:

> 爵士

should find:

> Jazz

Searching:

> sax

should find:

> Saxophone

Searching:

> 溫暖

should find relevant texture concepts.

---

33. DATABASE UX

Categories should be easy to scan.

Use compact cards or chips.

Each concept should clearly communicate:

What it is

and:

What it sounds like / does

without requiring music-production expertise.

Avoid displaying raw database metadata to the user.

---

34. NO AI API

The MVP must work without:

- OpenAI API
- Anthropic API
- Gemini API
- Grok API
- Suno API

Prompt generation is deterministic and local.

This makes the application:

- fast
- cheap
- private
- reliable
- usable without API configuration

---

35. PRIVACY

No user data should be sent to a server.

History and Favorites remain in LocalStorage.

No login is required.

---

36. ACCEPTANCE CRITERIA

The application is considered complete when all of the following work:

Core

- User can browse music concepts.
- User can search concepts.
- User can add concepts.
- User can remove concepts.
- User can expand Level 3 concepts.
- User can designate Primary Genre.
- Additional Genres become Influences.

- Prompt automatically updates.

Prompt Engine

- Prompt ordering is deterministic.
- Configurable weights are used.
- Primary Genre appears prominently.
- Influences are represented as influences.
- Categories are not blindly concatenated.
- Obvious redundancy is handled.
- Final prompt is natural English.

Conflict System

- Basic conflict rules exist.
- Warnings appear when appropriate.
- Warnings never block generation.
- User selections are never silently changed.

Output

- Structured preview works.
- Final prompt is editable.
- Reset works.
- Copy works.
- Copy feedback works.

Persistence

- History works.
- History is limited to 30 entries.
- Favorites work.
- Favorites can be named.
- Favorites can be edited.
- Favorites can be deleted.
- Data survives refresh.

Content

- All 30 genres exist.
- Each genre has useful representative data.
- Level 2 data is available.
- Level 3 expansion exists where appropriate.
- Traditional Chinese explanations exist.
- Music terminology follows the provided PDF.

UX

- No wizard.
- User can build a prompt quickly.
- Starter presets work.
- Desktop workspace is polished.
- Mobile layout remains usable.
- No unnecessary feature creep.

---

37. IMPORTANT PRODUCT DECISION

Do not turn this into an "AI music assistant."

The MVP is intentionally a **structured prompt-building instrument**.

The user supplies the creative direction.

The application supplies:

vocabulary + structure + musical relationships + ordering + convenience.

The system should help users become better at writing Suno prompts rather than hiding the process behind an AI black box.

---

38. FINAL IMPLEMENTATION INSTRUCTION

Build the application now.

Do not respond with only a product plan.

Actually implement the functional web application.

Populate the initial music database using the attached PDF as the primary music-knowledge reference.

Prioritize a working end-to-end experience over decorative polish.

The critical path is:

Browse → Select → Build → Organize → Preview → Copy → Save

Every part of this path should work before adding optional polish.